

SOULDE Studio USB2CAN转换模块使用说明书V2.3

版本更新说明

版本	更新内容
v1.0	第一次更新文档
v2.0	更新固件与硬件
v2.1	更新收发示例
v2.2	加入带壳版本图片
v2.3	修改一些bug

产品概述

SOULDE Studio USB2CAN转换模块是一款高性能、小体积的USB转CAN接口设备，具有以下特点：

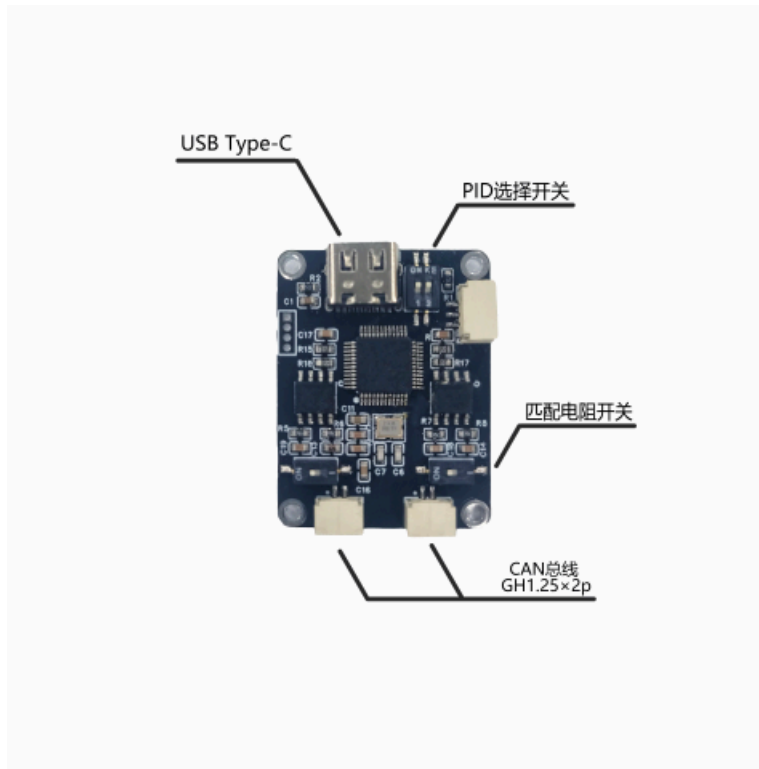
1. 小体积：模块尺寸仅为27mm × 35mm ，便于安装。
2. 高速传输：支持最高 1Mbps(CAN) 的数据传输速率，满足高速CAN总线通信需求。
3. USB虚拟串口：采用USB2.0 FS通讯协议，同时使用USB Type-C接口进行数据传输和供电。
4. 两路CAN接口：具备两个独立的CAN通道，可同时连接两个CAN网络。
5. 支持FDCAN：兼容FDCAN协议，适应更广泛的CAN总线应用。
6. 匹配电阻：模块为CAN总线设置了120 Ω匹配电阻，并设置拨码开关方便用户选择是否接入。
7. 设备名绑定：模块具有USB设备PID设置功能。通过拨码开关设置，可在Linux上将设备设置为 USB2CAN0-USB2CAN3 ，消除模块插入后设备名称变化以及多个模块之间的区分问题。

功能介绍

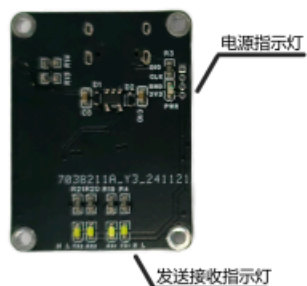
模块主要参数

使用说明

无壳版本



无壳版本，上图中，左侧为CAN1，右侧为CAN2，两路CAN线序均为左侧为CANL，右侧为CANH。



带壳版本



带壳版本，上图中，左侧为CAN2，右侧为CAN1，两路CAN线序均为左侧为CANH，右侧为CANL。
LED从左到右依次为CAN2接收、CAN2发送、CAN1发送、CAN1接收。

PID切换

用户可以使用PID切换开关切换设备PID，开关状态与自动识别的设备名称对应如下表(上表示拨到ON KE方向即朝USB口方向，下表示拨到 1 2方向)

开关1	开关2	设备名称
上	上	USB2CAN0
下	上	USB2CAN1
上	下	USB2CAN2
下	下	USB2CAN3

匹配电阻

模块为CAN总线设置了120Ω匹配电阻，并设置拨码开关方便用户选择是否接入，将CAN接口附近对应单路拨码开关设置为ON即接入匹配电阻。电阻默认接入。

LED指示灯

每一路CAN总线配有一对数据传输指示灯，数据传输时LED灯亮白光，无数据传输时LED灯熄灭。

驱动安装

目前，本模块支持在Linux系统中进行驱动安装和二次开发，未来将支持Windows。

安装至系统目录

在SDK包内，有 `install.sh` 用于系统级安装，将会把动态链接库安装至 `\usr\local\lib`，将头文件安装至 `\usr\local\include`，并自动安装 `usb_can.rules` 至 `\etc\udev\rules.d`。

同时，有 `uninstall.sh` 可以自动将安装的文件从系统中移除。

手动安装

如不希望使用脚本自动安装至系统，可以将 `lib` 文件夹中的 `libusb_can.so` 复制至工程，并手动设置动态链接，如Cmake工具链往CMakeLists.txt中添加

```
link_directories(DIR\CONTAINS\SHARELIB)
target_link_directories(${YOUR_TARGET} usb_can)
```

同时，将 `include` 文件夹中的 `usb_can.h` 复制至工程，使工程可以发现此头文件，如Cmake工具链往CMakeLists.txt中添加

```
include_directories(DIR\CONTAINS\HEADER)
```

要使用自动绑定设备名称功能，需要将 `usb_can.rules` 移动至 `\etc\udev\rules.d`，并使udev规则生效：

```
sudo udevadm trigger
```

二次开发说明

接口说明

目前，模块提供C/C++接口进行模块操作，下面给出模块操作的主要结构体和接口函数

```

#define STANDARD 0
#define EXTENDED 1

typedef struct FrameInfo {
    uint32_t canID; // CAN帧ID
    uint8_t frameType; // 0为标准帧, 1为扩展帧, 推荐使用预先定义的宏
    uint8_t dataLength; //CAN帧数据长度, 范围0-8
} FrameInfo;

//!
//! \param devName 设备名称, 如 '/dev/USB2CAN0'
//! \return 设备句柄, 用于操作设备进行收发, -1表示打开失败
int32_t openUSBCAN(const char *devName);

//!
//! \param dev 设备句柄
//! \return 串口关闭返回值, 0表示成功, -1表示失败
int32_t closeUSBCAN(int32_t dev);

//!
//! \param dev 设备句柄
//! \param channel USB2CAN 模块的CAN通道, USB2CAN-CAN-Dual可设置为1或2
//! \param info FrameInfo结构体指针, 用于设置CAN帧ID、帧类型和数据长度
//! \param data CAN帧数据
//! \return 传输成功返回数据长度, -1表示失败
int32_t sendUSBCAN(int32_t dev, uint8_t channel, FrameInfo *info, uint8_t *data);

//!
//! \param dev 设备句柄
//! \param channel USB2CAN 收到数据的CAN通道, 需要提前定义变量并传入指针
//! \param info FrameInfo结构体指针, 需要提前定义变量, 用于接受CAN帧ID、帧类型和数据长度
//! \param data CAN帧数据, 需要提前定义变量, 用于接受CAN帧数据
//! \param timeout 接收超时时间, 单位为微妙。函数会阻塞等待, 直到接收到数据或者超时。
//! \return 返回0表示接收成功, 返回-1表示接收超时
int32_t readUSBCAN(int32_t dev, uint8_t *channel, FrameInfo *info, uint8_t *data, int32_t timeout);

```

使用示例

下面给出一个简单的使用示例, 用于演示如何使用上述接口函数进行模块操作

发送CAN帧

```
#include <stdio.h>
#include <stdint.h>
#include "usb_can.h"

int main() {
    int32_t dev = openUSBCAN("/dev/USB2CAN0"); //打开USB2CAN0模块
    if (dev < 0) {
        printf("Failed to open USB2CAN device\n");
        return -1;
    }

    FrameInfo info;
    info.canID = 0x123; // 设置CAN ID 为0x123
    info.frameType = STANDARD; //设置为标准帧
    info.dataLength = 8; //设置数据长度为8
    uint8_t data[8] = {0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08};

    int32_t ret = sendUSBCAN(dev, 1, &info, data); // 从模块CAN1发送CAN帧
    if (ret != 8) {
        printf("Failed to send CAN frame\n");
        closeUSBCAN(dev);
        return -1;
    }
}
```

接受CAN帧

```
#include <stdio.h>
#include <stdint.h>
#include "usb_can.h"

int main() {
    int32_t dev = openUSBCAN("/dev/USB2CAN0"); //打开USB2CAN0模块
    if (dev < 0) {
        printf("Failed to open USB2CAN device\n");
        return -1;
    }

    uint8_t channel; //初始化接收到的CAN通道
    FrameInfo info; //初始化接收到的CAN帧信息
    uint8_t data[8]; //初始化接收到的CAN帧数据

    ret = readUSBCAN(dev, &channel, &info, data, 1e6); // 从模块CAN1接收CAN帧，超时时间为1s
    if (ret == 0) {
        printf("Received CAN frame on channel %d\n", channel);
        printf("CAN ID: 0x%03X\n", info.canID);
        printf("Frame type: %s\n", info.frameType == STANDARD ? "Standard" : "Extended");
    }
    closeUSBCAN(dev);
    return 0;
}
```

售后服务

SOULDE Studio提供完善的售后服务，如有任何疑问或需求，请随时与我们联系。我们将竭诚为您解答和解决问题。模块出现非人为损坏质量问题，可以与店铺联系退换，来回邮费由我们承担。

感谢您选择SOULDE Studio USB2CAN转换模块，祝您使用愉快！